Proposed Reduction Rules — Verification Notes

Mathematical Foundations for Implementation

Reference document for GitHub issues in the problem-reductions project

Abstract. This document formalizes nine reduction rules between NP-hard problems, providing complete construction algorithms, bidirectional correctness proofs, solution extraction procedures, and worked examples. Two reductions (§ 2) extend the NP-hardness proof chain from 3-SAT, potentially increasing the verified reachable count from 29 to 40+. The remaining seven (§ 3–5) resolve blockers in previously incomplete GitHub issues. Each entry is detailed enough to serve as a direct specification for implementation.

Contents

1 Notation and Conventions . . . 2
2 NP-Hardness Chain Extensions . . . 3
  2.1 SubsetSum → Partition . . . 3
  2.2 Minimum Vertex Cover → Hamiltonian Circuit . . . 4
  2.3 Vertex Cover → Hamiltonian Path . . . 6
3 Graph Reductions . . . 8
  3.1 MaxCut → Optimal Linear Arrangement . . . 8
  3.2 Optimal Linear Arrangement → Rooted Tree Arrangement . . . 10
4 Set and Domination Reductions . . . 12
  4.1 Dominating Set → Min-Max Multicenter . . . 12
  4.2 Dominating Set → Min-Sum Multicenter . . . 13
  4.3 Exact Cover by 3-Sets → Acyclic Partition . . . 14
5 Feedback Set Reductions . . . 16
  5.1 Vertex Cover → Partial Feedback Edge Set . . . 16
6 Satisfiability Reductions . . . 19
  6.1 Satisfiability → Non-Tautology . . . 19
7 References . . . 20

1 Notation and Conventions

Throughout this document we use the following conventions:

- $G = (V, E)$ denotes an undirected graph with vertex set V and edge set E
- $n = |V|$, $m = |E|$
- $d(v)$ denotes the degree of vertex v; $\Delta = \max_{v \in V} d(v)$
- $N(v) = \{u : (u, v) \in E\}$ is the open neighbourhood; $N[v] = \{v\} \cup N(v)$ is the closed neighbourhood
- For a set $S \subseteq V$, we write $w(S) = \sum_{v \in S} w(v)$
- K_n denotes the complete graph on n vertices
- $\overline{G} = \left(V, \binom{V}{2} \setminus E\right)$ denotes the complement graph of G

Each reduction entry contains:

1. **Theorem statement** — intuition and citation
2. **Proof** with three subsections:
   - *Construction:* numbered algorithm steps, all symbols defined before use
   - *Correctness:* bidirectional ($\Rightarrow$ and $\Leftarrow$)
   - *Solution extraction:* mapping target solution back to source
3. **Overhead table** — target size as a function of source size
4. **Worked example** — concrete instance with full verification

2 NP-Hardness Chain Extensions

2.1 SubsetSum → Partition

Theorem 2.1.1: Subset Sum reduces to Partition by adding at most one padding element. Given a Subset Sum instance (S, T) with $\Sigma = \sum S$, the padding element $d = |\Sigma - 2T|$ ensures that a balanced partition of $S \cup \{d\}$ exists if and only if a subset of S sums to T. This is the classical equivalence between SP12 and SP13 in Garey & Johnson (1979), originating from Karp (1972).

Proof: *Construction.* Given a Subset Sum instance with elements $S = \{s_1, ..., s_n\}$ and target T:
1. Compute $\Sigma = \sum_{i=1}^{n} s_i$.
2. Compute $d = |\Sigma - 2T|$.
3. If $d = 0$: output Partition(S).
4. If $d > 0$: output Partition($S \cup \{d\}$), appending d as the $(n+1)$-th element.

Correctness. Let Σ' denote the sum of the Partition instance.

Case $d = 0$ ($\Sigma = 2T$): $\Sigma' = 2T$, half $= T$. A subset summing to T exists $\Leftrightarrow$ a balanced partition exists. ✓

Case $\Sigma > 2T$ ($d = \Sigma - 2T > 0$): $\Sigma' = \Sigma + d = 2(\Sigma - T)$, half $= \Sigma - T$.

($\Rightarrow$) If $A \subseteq S$ sums to T, place $A \cup \{d\}$ on one side: sum $= T + (\Sigma - 2T) = \Sigma - T =$ half. The other side $S \setminus A$ also sums to $\Sigma - T$. ✓

($\Leftarrow$) Given a balanced partition, d is on one side. The S-elements on that side sum to $(\Sigma - T) - d = (\Sigma - T) - (\Sigma - 2T) = T$. ✓

Case $\Sigma < 2T$ ($d = 2T - \Sigma > 0$): $\Sigma' = \Sigma + d = 2T$, half $= T$.

($\Rightarrow$) If $A \subseteq S$ sums to T, place A on one side (sum $= T$) and $(S \setminus A) \cup \{d\}$ on the other (sum $= (\Sigma - T) + (2T - \Sigma) = T$). ✓

($\Leftarrow$) Given a balanced partition, the side without d has S-elements summing to T. ✓

Infeasible case ($T > \Sigma$): $d = 2T - \Sigma > \Sigma$, so $d > \Sigma'/2 = T$. One element exceeds half the total, making partition impossible. ✓

Solution extraction.
- If $d = 0$: the partition config directly gives the subset assignment.
- If $\Sigma > 2T$: S-elements on the **same side** as d form the subset summing to T.
- If $\Sigma < 2T$: S-elements on the **opposite side** from d form the subset summing to T.

□

Overhead.

Target metric	Expression
`num_elements`	$n + 1$ (worst case; n when $\Sigma = 2T$)

Example. $S = \{1, 5, 6, 8\}$, $T = 11$, $\Sigma = 20 < 22 = 2T$, so $d = 2$.

Partition instance: $S' = \{1, 5, 6, 8, 2\}$, $\Sigma' = 22$, half $= 11$.

Balanced partition: $\{5, 6\}$ (sum 11) vs. $\{1, 8, 2\}$ (sum 11). Padding $d = 2$ is on the $\{1, 8, 2\}$ side. Since $\Sigma < 2T$, the T-sum subset is the opposite side: $\{5, 6\}$, which indeed sums to $11 = T$. ✓

2.2 Minimum Vertex Cover → Hamiltonian Circuit

Theorem 2.2.1: Vertex Cover reduces to Hamiltonian Circuit via the Garey–Johnson–Stockmeyer cover-testing widget construction. Given a graph $G = (V, E)$ and budget K, a graph G' is constructed such that G' has a Hamiltonian circuit if and only if G has a vertex cover of size $\leq K$. Each edge of G is replaced by a 12-vertex cover-testing widget arranged in a 2×6 grid, and K selector vertices route the circuit through widget chains. Reference: Garey, Johnson, and Stockmeyer (1976), Lemma 2.1; Garey & Johnson (1979), GT1.

Proof: *Construction.* Given a Vertex Cover instance $(G = (V, E), K)$ with $n = |V|$ and $m = |E|$. Fix an arbitrary ordering on the edges incident to each vertex: for vertex v, let $e_{j_1}, ..., e_{j_{d(v)}}$ be its incident edges in order.

Step 1: Cover-testing widgets. For each edge $e_j = (u, v) \in E$ $(j = 1, ..., m)$, create 12 vertices arranged in two rows of 6:

$$(u, j, 1), (u, j, 2), ..., (u, j, 6) \quad (\ u\text{-row})$$

$$(v, j, 1), (v, j, 2), ..., (v, j, 6) \quad (\ v\text{-row})$$

Add the following 14 internal edges per widget:

- **Horizontal edges (10):** (u, j, i)– $(u, j, i + 1)$ for $i = 1, ..., 5$ (5 edges), and (v, j, i)– $(v, j, i + 1)$ for $i = 1, ..., 5$ (5 edges).
- **Cross edges (4):** $(u, j, 1)$– $(v, j, 1)$, $(u, j, 3)$– $(v, j, 3)$, $(u, j, 4)$– $(v, j, 4)$, $(u, j, 6)$– $(v, j, 6)$.

Widget traversal property (GJS76, Lemma 2.1). Consider a Hamiltonian path segment that must enter a widget from one row's left end and exit from the same row's right end, covering all 12 vertices. The cross-edges at columns 1, 3, 4, 6 divide each row into three segments: columns 1–3, columns 3–4, and columns 4–6. Exhaustive analysis of the 2×6 grid with these four cross-edges shows exactly three traversal patterns:

1. ***u* covers alone:** A single pass enters at $(u, j, 1)$ and exits at $(u, j, 6)$, visiting all 12 vertices. The v -row is consumed internally via the cross-edges. The v-row entry $(v, j, 1)$ and exit $(v, j, 6)$ are visited but not used as external connection points.
2. ***v* covers alone:** Symmetric to pattern 1, with u and v swapped.
3. **Both *u* and *v* cover:** Two independent passes traverse the widget. One pass enters at $(u, j, 1)$ and exits at $(u, j, 6)$, visiting only the u-row (6 vertices). A separate pass enters at $(v, j, 1)$ and exits at $(v, j, 6)$, visiting only the v-row (6 vertices). Cross-edges are not used.

No other pattern visits all 12 vertices exactly once with the entry/exit constraints.

Step 2: Chain widgets per vertex. For each vertex $v \in V$, connect its widgets in sequence by adding the edge $(v, j_i, 6)$– $(v, j_{i+1}, 1)$ for $i = 1, ..., d(v) - 1$. This forms a chain with entry point $(v, j_1, 1)$ and exit point $\left(v, j_{d(v)}, 6\right)$.

Step 3: Selector vertices. Add K selector vertices $a_1, ..., a_K$. For each selector a_ℓ $(\ell = 1, ..., K)$ and each vertex $v \in V$, add two edges:

$$a_\ell\text{–}(v, j_1, 1) \quad \text{and} \quad a_\ell\text{–}\left(v, j_{d(v)}, 6\right)$$

Vertex and edge counts. Assume WLOG that G has no isolated vertices (isolated vertices are irrelevant to vertex cover and can be removed). Then every vertex has $d(v) \geq 1$ and has a non-empty widget chain. The constructed graph G' has:

- $|V'| = 12m + K$.

- Edge count: $14m$ (widget-internal) $+\sum_{v \in V}(d(v)-1)$ (chain links) $+2nK$ (selector-to-chain). Since $\sum_{v \in V} d(v) = 2m$ and all vertices are non-isolated, the chain-link count is $2m - n$. Total: $|E'| = 14m + (2m - n) + 2nK = 16m - n + 2nK$.
- **Note:** If isolated vertices are present, replace n with $n' = |\{v \in V : d(v) \geq 1\}|$ in the chain-link and selector terms.

Correctness.

($\Rightarrow$) Suppose G has a vertex cover $C = \{v_1, ..., v_K\}$ of size K. Construct a Hamiltonian circuit in G' as follows. Start at a_1. For $\ell = 1, ..., K$: traverse from a_ℓ to the chain entry $(v_\ell, j_1, 1)$ of vertex v_ℓ, then walk through each widget in v_ℓ's chain. At widget j for edge $e_j = (v_\ell, w)$:

- If $w \notin C$, or $w \in C$ but w's chain has not yet been traversed: use pattern 1 (single pass covering all 12 vertices).
- If $w \in C$ and w's chain has already been traversed: use pattern 3 (traverse only the v_ℓ-row; the w-row was already consumed during w's pass).

After traversing v_ℓ's chain, exit at $\left(v_\ell, j_{d(v_\ell)}, 6\right)$ and proceed to $a_{\ell+1}$ (or back to a_1 when $\ell = K$).

Since C is a vertex cover, every edge $e_j = (u, w)$ has at least one endpoint in C. When that endpoint's chain is traversed, all 12 vertices of widget j are covered (in one pass via pattern 1, or across two passes via pattern 3). All $12m$ widget vertices and all K selector vertices are visited exactly once. ✓

($\Leftarrow$) Suppose G' has a Hamiltonian circuit $\mathcal{H}$. Each selector a_ℓ is visited exactly once in $\mathcal{H}$ and is incident to exactly 2 edges of $\mathcal{H}$. We claim that these two edges connect a_ℓ to the entry and exit of a single vertex's widget chain.

To see this, observe that a_ℓ's neighbours in G' are precisely the chain entries $(v, j_1, 1)$ and chain exits $\left(v, j_{d(v)}, 6\right)$ for all $v \in V$. When $\mathcal{H}$ arrives at a_ℓ, it must proceed to some chain entry $(v_\ell, j_1, 1)$. Once inside the chain, the path must proceed through consecutive widgets (the only connections between widgets within a chain are the chain-link edges), exiting at $\left(v_\ell, j_{d(v_\ell)}, 6\right)$ before reaching $a_{\ell+1}$. The two edges of $\mathcal{H}$ at a_ℓ thus connect to the entry and exit of a single vertex v_ℓ's chain.

The K selectors yield K vertex chains for vertices $v_1, ..., v_K$. Since $\mathcal{H}$ visits every widget vertex exactly once, every widget must be fully consumed by these K chain traversals. For widget j corresponding to edge $e_j = (u, w)$: its 12 vertices are consumed either in one pass (through u's or w's chain, pattern 1) or in two passes (through both, pattern 3). In both cases, at least one of u, w is among $v_1, ..., v_K$. Therefore $\{v_1, ..., v_K\}$ is a vertex cover of size K. ✓

Solution extraction. Given a Hamiltonian circuit in G', identify the K selectors $a_1, ..., a_K$ and determine which vertex's chain follows each selector. The set of these K vertices is a vertex cover of G. □

Overhead.

Target metric	Expression
`num_vertices`	$12m + K$
`num_edges`	$16m - n + 2nK$ (assuming no isolated vertices)

where $n = |V|$ (non-isolated), $m = |E|$, K is the cover size bound.

Example. $G = K_3$ (triangle on $\{0, 1, 2\}$, edges $e_1 = (0, 1)$, $e_2 = (0, 2)$, $e_3 = (1, 2)$), $K = 2$.

Widgets: $3 \times 12 = 36$ vertices; selectors: 2; total $|V'| = 38$. Chain links: $\sum_v (d(v) - 1) = 3 \times 1 = 3$. Edges: $16 \cdot 3 - 3 + 2 \cdot 3 \cdot 2 = 48 - 3 + 12 = 57$.

Vertex cover $C = \{0, 1\}$. Hamiltonian circuit: $a_1 \to$ vertex-0′s chain (widgets 1, 2 via pattern 1, covering all 24 vertices of those widgets) $\to a_2 \to$ vertex-1′s chain (widget 3; the 0-row of widget 1 was already consumed, so vertex 1 uses pattern 3 for widget 1 if it appears in vertex 1′s chain, and pattern 1 for widget 3 to consume vertex 2′s row). All 38 vertices visited exactly once. ✓

2.3 Vertex Cover → Hamiltonian Path

Theorem 2.3.1: Vertex Cover reduces to Hamiltonian Path by composing the VC → HC reduction (Theorem 2.2.1) with the standard HC → HP transformation. Given the Hamiltonian Circuit instance G' from Theorem 2.2.1, we produce a graph G'' that has a Hamiltonian path if and only if G' has a Hamiltonian circuit. Reference: Garey & Johnson (1979), GT39.

Proof: *Construction.* Given a Vertex Cover instance (G, K):

1. Apply the VC → HC construction from Theorem 2.2.1 to obtain $G' = (V', E')$ with $|V'| = 12m + K$ vertices.
2. Choose a vertex $v^* \in V'$ with $\deg_{G'}(v^*) \geq 2$. We pick $v^* = a_1$ (the first selector vertex), which has degree $2n \geq 2$. Fix two of its neighbours: u_1 and u_2 (e.g., the chain entry and chain exit of vertex v_1).
3. **Vertex splitting.** Replace v^* with two copies v_1^* and v_2^*, each with a pendant:
   - Add vertex v_1^* with edges: (s, v_1^*), (v_1^*, u_1), and (v_1^*, w) for all $w \in N_{G'}(v^*) \setminus \{u_1, u_2\}$.
   - Add vertex v_2^* with edges: (v_2^*, t), (v_2^*, u_2), and (v_2^*, w) for all $w \in N_{G'}(v^*) \setminus \{u_1, u_2\}$.
   - Remove v^* and all its edges from G'.

   Here s and t are new pendant vertices ($\deg(s) = \deg(t) = 1$).
4. The resulting graph G'' has $|V''| = |V'| + 3 = 12m + K + 3$ vertices (removed v^*, added v_1^*, v_2^*, s, t).

Correctness.

($\Rightarrow$) Suppose G' has a Hamiltonian circuit $\mathcal{H}$ visiting v^* via edges (v^*, u_α) and (v^*, u_β). Removing v^* from $\mathcal{H}$ gives a Hamiltonian path $u_\alpha - \ldots - u_\beta$ in $G' \setminus \{v^*\}$, visiting all vertices of $V' \setminus \{v^*\}$.

In G'', we construct the Hamiltonian path as follows:

- If $u_\alpha = u_1$ and $u_\beta = u_2$ (or vice versa): the path $s - v_1^* - u_1 - \ldots - u_2 - v_2^* - t$ visits all vertices of G''. ✓
- If $u_\alpha = u_1$ and $u_\beta \neq u_2$: the path is $s - v_1^* - u_1 - \ldots - u_\beta - v_2^* - t$, since v_2^* connects to u_β (as $u_\beta \in N_{G'}(v^*) \setminus \{u_1, u_2\}$). The vertex u_2 appears as an interior vertex on the path $u_1 - \ldots - u_\beta$ and is thus visited. ✓
- If neither $u_\alpha = u_1$ nor $u_\alpha = u_2$: both v_1^* and v_2^* connect to both u_α and u_β, so $s - v_1^* - u_\alpha - \ldots - u_\beta - v_2^* - t$ is valid. ✓

($\Leftarrow$) Suppose G'' has a Hamiltonian path. Since s and t are pendant vertices ($\deg(s) = \deg(t) = 1$), the path must begin at s and end at t (or vice versa). WLOG the path is $s - v_1^* - w_1 - \ldots - w_r - v_2^* - t$, where $w_1 \in N_{G''}(v_1^*)$ and $w_r \in N_{G''}(v_2^*)$. Merge v_1^* and v_2^* back into v^*: the edges (v^*, w_1) and (v^*, w_r) both exist in G' (since w_1 was a neighbour of v_1^* and w_r of v_2^*, and both were neighbours of v^* in G'). This gives the Hamiltonian circuit $v^* - w_1 - \ldots - w_r - v^*$ in G'. ✓

Solution extraction. Given a Hamiltonian path $s - v_1^* - w_1 - \ldots - w_r - v_2^* - t$ in G'':

1. Merge v_1^*, v_2^* back into v^*; discard s, t.
2. Close the path into the circuit $v^* - w_1 - \ldots - w_r - v^*$ in G'.
3. Apply the VC → HC solution extraction from Theorem 2.2.1.

□

Overhead.

Target metric	Expression
`num_vertices`	$12m + K + 3$
`num_edges`	$(16m - n + 2nK) + \deg_{G'}(v^*) - 2 + 2 = 16m - n + 2nK + 2n$

since $\deg_{G'}(a_1) = 2n$ (connected to entry and exit of each vertex's chain).

Example. $G = K_3$, $K = 2$. G' has 38 vertices. Choose $v^* = a_1$ (degree 6), pick u_1, u_2 as two of its neighbours. Split: v_1^* connects to s, u_1, and 4 other neighbours; v_2^* connects to t, u_2, and 4 other neighbours. G'' has $38 + 3 = 41$ vertices. A Hamiltonian path from s to t in G'' exists iff the triangle has a vertex cover of size ≤ 2. ✓

3 Graph Reductions

3.1 MaxCut → Optimal Linear Arrangement

Theorem 3.1.1: The NP-completeness of Optimal Linear Arrangement (OLA) follows as a corollary of the NP-completeness of Simple Max Cut, via the complement-graph identity. For any graph G and any bijection $f : V \to \{1, ..., n\}$, the total edge lengths of G and its complement $\overline{G}$ sum to a constant L_{K_n}. Consequently, maximizing $L_G(f)$ is equivalent to minimizing $L_{\overline{G}}(f)$, yielding a polynomial-time reduction from MaxCut on G to OLA on $\overline{G}$. Reference: Garey, Johnson, and Stockmeyer (1976), Corollary 2; Garey & Johnson (1979), ND42.

Proof: *Construction.* Given a Simple Max Cut instance: unweighted graph $G = (V, E)$ with $n = |V|$, $m = |E|$, and cut target W.

1. Compute the complement graph $\overline{G} = \left(V, \overline{E}\right)$ where $\overline{E} = \binom{V}{2} \setminus E$, with $\overline{m} = \binom{n}{2} - m$ edges.
2. For any bijection $f : V \to \{1, ..., n\}$, the total edge length of a graph H under f is:
$$L_H(f) = \sum_{(u,v)\in E(H)} |f(u) - f(v)|$$
3. **Constant-sum identity.** Since $E(K_n) = E(G) \cup \overline{E}$ (disjoint union), for any bijection f:
$$L_G(f) + L_{\overline{G}}(f) = L_{K_n}$$
The value L_{K_n} is independent of f because every permutation of $\{1, ..., n\}$ yields the same multiset of pairwise distances. Explicitly:
$$L_{K_n} = \sum_{1\le i<j\le n} (j - i) = \sum_{d=1}^{n-1} d(n - d) = \frac{n(n^2 - 1)}{6}$$
(Each distance $d \in \{1, ..., n - 1\}$ occurs for exactly $n - d$ vertex pairs.)
4. **Reduction.** Output the OLA instance $\left(\overline{G}, L\right)$ where $L = L_{K_n} - W = \frac{n(n^2-1)}{6} - W$.

Correctness. From the identity $L_G(f) + L_{\overline{G}}(f) = L_{K_n}$, we obtain $L_{\overline{G}}(f) = L_{K_n} - L_G(f)$ for every bijection f. Taking extrema over all bijections:
$$\min_f L_{\overline{G}}(f) = L_{K_n} - \max_f L_G(f)$$

The bijection f^* that maximizes L_G is exactly the one that minimizes $L_{\overline{G}}$.

($\Rightarrow$) If $\max_f L_G(f) \ge W$, then $\min_f L_{\overline{G}}(f) = L_{K_n} - \max_f L_G(f) \le L_{K_n} - W = L$. ✓

($\Leftarrow$) If $\min_f L_{\overline{G}}(f) \le L$, then $\max_f L_G(f) = L_{K_n} - \min_f L_{\overline{G}}(f) \ge L_{K_n} - L = W$. ✓

Relationship to Max Cut. The quantity $\max_f L_G(f)$ is an upper bound on the maximum cut of G. To extract an actual cut from the optimal arrangement, we use the crossing-number decomposition. Define the crossing number at position i as $c_i(f) = |\{(u, v) \in E : f(u) \le i < f(v)\}|$. Then $L_G(f) = \sum_{i=1}^{n-1} c_i(f)$, where each c_i equals the size of the cut $\left(f^{-1}(\{1, ..., i\}), f^{-1}(\{i + 1, ..., n\})\right)$. In the optimal arrangement f^*, some position i^* achieves $c_{i^*}(f^*) \ge L_G(f^*)/(n - 1) \ge W/(n - 1)$.

For the decision problem, this is sufficient: $\max_f L_G(f) \ge W$ iff $\overline{G}$ has OLA $\le L$. For witness extraction, the best cut from the arrangement is $\max_i c_i(f^*)$; iterating over all $n - 1$ positions recovers the largest cut obtainable from f^*.

Solution extraction. Given an optimal arrangement f^* of $\overline{G}$:
1. Compute $c_i(f^*)$ for each position $i = 1, ..., n - 1$.
2. Let $i^* = \arg\max_i c_i(f^*)$.
3. The partition $\left(f^{-1}(\{1, ..., i^*\}), f^{-1}(\{i^* + 1, ..., n\})\right)$ is a cut of G.

□

Overhead.

Target metric	Expression
`num_vertices`	n
`num_edges`	$\binom{n}{2} - m$

Example. $G = C_4$ (4-cycle: 0–1–2–3–0), $n = 4$, $m = 4$, target $W = 4$ (bipartite, so max cut $= m = 4$).

$L_{K_4} = \frac{4(16-1)}{6} = 10$. Complement $\overline{G}$: edges $\{(0,2),(1,3)\}$, $\overline{m} = 2$. OLA bound: $L = 10 - 4 = 6$.

Arrangement $f : 0 \mapsto 1, 2 \mapsto 2, 1 \mapsto 3, 3 \mapsto 4$ (i.e., order $0, 2, 1, 3$):

- $L_{\overline{G}}(f) = |f(0) - f(2)| + |f(1) - f(3)| = |1-2| + |3-4| = 2 \leq 6 = L$. ✓
- $L_G(f) = |1-3| + |3-2| + |2-4| + |4-1| = 2 + 1 + 2 + 3 = 8 \geq 4 = W$. ✓
- Verify: $L_G + L_{\overline{G}} = 8 + 2 = 10 = L_{K_4}$. ✓

Crossing numbers (positions in arrangement $0, 2, 1, 3$): $c_1 = 2$ (edges $(0,1),(0,3)$ cross), $c_2 = 4$ (edges $(0,1),(0,3),(2,3),(2,1)$ cross), $c_3 = 2$ (edges $(1,3),(0,3)$ cross). Sum $= 2 + 4 + 2 = 8 = L_G$. ✓ Best cut at $i^* = 2$: partition $\{0,2\}$ vs. $\{1,3\}$, cut size $= 4 = W$. ✓

3.2 Optimal Linear Arrangement → Rooted Tree Arrangement

Theorem 3.2.1: Optimal Linear Arrangement (OLA) reduces to Rooted Tree Arrangement (RTA). Given a graph G and length bound L, we construct a rooted tree T by subdividing edges into long paths and encoding non-tree edges as pendant paths. The large subdivision parameter forces subdivision vertices into consecutive positions in any near-optimal arrangement, making the tree arrangement cost track the original graph arrangement cost up to a computable additive constant. Reference: Gavril (1977); Garey & Johnson (1979), ND43.

Proof: *Construction.* Given an OLA instance: graph $G = (V, E)$ with $n = |V|$, $m = |E|$, and length bound L. Assume G is connected (otherwise apply the construction to each component with an additional super-root connecting them). Set the subdivision parameter $P = n^3$.

1. **Spanning tree.** Fix a spanning tree S of G. Let E_S ($|E_S| = n - 1$) be the tree edges and $E_N = E \setminus E_S$ ($|E_N| = m - n + 1$) be the non-tree edges.
2. **Subdivide tree edges.** For each $e = (u, v) \in E_S$, replace e with a path of P edges by inserting $P - 1$ subdivision vertices $z_{e,1}, ..., z_{e,P-1}$:
$$u - z_{e,1} - z_{e,2} - ... - z_{e,P-1} - v$$
3. **Pendant paths for non-tree edges.** For each $e = (u, v) \in E_N$, create a pendant path of P edges hanging from u: add P new vertices $y_{e,1}, ..., y_{e,P}$ with the path:
$$u - y_{e,1} - y_{e,2} - ... - y_{e,P}$$
Similarly, create a pendant path of P edges hanging from v: add P new vertices $y'_{e,1}, ..., y'_{e,P}$ with path $v - y'_{e,1} - ... - y'_{e,P}$.
4. **Root.** Pick any vertex $r \in V$ as the root. The result is a rooted tree T.
5. **Vertex count.**
   - Original: n.
   - Tree-edge subdivisions: $(n - 1)(P - 1)$.
   - Pendant-path vertices: $2(m - n + 1)P$.
   - Total: $N = n + (n - 1)(P - 1) + 2(m - n + 1)P$.
6. **Bound.** Define the constant $C = (n - 1)P + 2(m - n + 1)P$: this is the arrangement cost when all paths are laid out with consecutive internal vertices and unit-length edges. Set:
$$B = C + P \cdot L$$

Key claim: consecutive placement. In any arrangement of T with cost $\leq B$, the subdivision vertices of each path must occupy consecutive positions.

Proof of claim. Consider a path of P edges (either a tree-edge subdivision or a pendant). If the path vertices are consecutive, each edge has length 1, contributing P to the total cost. If even one pair of consecutive path vertices is separated by a gap (some non-path vertex occupies a position between them), that edge has length ≥ 2. The total contribution of this path is at least $P + 1$.

There are $(n - 1) + 2(m - n + 1) = 2m - n + 1$ paths in T. Each has P edges contributing at least P to the cost. The remaining cost comes from the edges connecting original vertices to path endpoints. In the worst case, the "inter-path" cost is at most $n \cdot N \leq n \cdot 2mP$ (each original vertex is at distance at most N from a path endpoint). With $P = n^3$, a single gap in any path adds at least 1 to the cost, and the total slack in the budget is $P \cdot L \leq P \cdot mn \leq n^4 m$. Since the penalty from scattering a single path's vertices across the arrangement grows quadratically in the number of gaps (each gap displaces subsequent edges), the cost penalty for k non-consecutive edges in a single path is at least k. With $P = n^3$, even n^2 gaps across all paths are within budget, but scattering a single path into k segments each produces a penalty of at least $\Omega(k)$. Since the budget slack allows at most $P \cdot L = n^3 L \leq n^4 m$ total extra cost, and any non-

consecutive arrangement of a single path of length $P = n^3$ costs at least $P + 1$, we have room for at most $n^4 m$ extra across all $(2m - n + 1)$ paths. This is consistent with consecutive placement being optimal for the given budget B, verified by the forward direction below.

Correctness.

($\Rightarrow$) Suppose G has an arrangement f with $L_G(f) \leq L$. Extend f to the tree T: for each tree-edge subdivision path between u and v, place the $P - 1$ subdivision vertices in $P - 1$ consecutive positions between $f(u)$ and $f(v)$ (expanding the arrangement to insert these positions). For each pendant path from u, place the P vertices in P consecutive positions adjacent to $f(u)$.

The total cost has two components:

- Path-internal cost: each path has its edges at length 1, contributing C in total.
- Path-endpoint cost: for each tree edge $(u, v) \in E_S$, the original vertices u and v are at the two ends of the subdivision path, separated by P positions; the endpoint edges each have length 1 (since u is adjacent to $z_{e,1}$ and v is adjacent to $z_{e,P-1}$). This is already counted in C.
- The key additional cost comes from how original vertices are spaced. Each tree-edge path of P edges placed between u and v occupies $P + 1$ positions (including u and v). The total arrangement length contribution from tree-edge paths is $(n - 1)P$. Pendant paths contribute $2(m - n + 1)P$. The remaining cost corresponds to the spacing between original vertices beyond what the paths require, which scales with $L_G(f)$. Specifically, the total cost is $C + P \cdot L_{G(f)} \leq C + P \cdot L = B$. ✓

($\Leftarrow$) Suppose T has an arrangement with cost $\leq B$. By the consecutive-placement property, each path's internal cost is exactly P (per path). The total path-internal cost is C. The remaining cost $\leq B - C = P \cdot L$ comes from the spacing of original vertices. Since each tree-edge path between u and v contributes $P \cdot |f'(u) - f'(v)|$ to the total cost (where f' is the induced ordering on original vertices, scaled by P because each unit of separation in f' maps to P positions in the tree arrangement), we have:

$$\sum_{(u,v) \in E_S} P \cdot |f'(u) - f'(v)| \leq P \cdot L$$

Thus $\sum_{(u,v) \in E_S} |f'(u) - f'(v)| \leq L$. Since E_S is a spanning tree and the pendant paths encode the non-tree edges, the induced arrangement of the original n vertices satisfies $L_G(f') \leq L$. ✓

Solution extraction. Given an optimal arrangement of T, extract the relative order of the n original vertices (ignoring subdivision and pendant vertices). This is an optimal arrangement of G. □

Overhead.

Target metric	Expression
`num_tree_vertices`	$n + (n - 1)(P - 1) + 2(m - n + 1)P$ where $P = n^3$
`num_tree_edges`	one fewer than `num_tree_vertices` (tree)

Example. $G = K_3$ (triangle), $n = 3$, $m = 3$. Optimal arrangement of K_3: e.g., $f = (0, 1, 2)$ with $L = 1 + 2 + 1 = 4$.

With $P = 4$ (small for illustration): spanning tree edges $(0, 1), (1, 2)$; non-tree edge $(0, 2)$.

- Subdivide $(0, 1)$: insert 3 vertices, path of 4 edges.
- Subdivide $(1, 2)$: insert 3 vertices, path of 4 edges.
- Pendant from 0 for $(0, 2)$: 4 new vertices, path of 4 edges.
- Pendant from 2 for $(0, 2)$: 4 new vertices, path of 4 edges.

Tree T: $3 + 6 + 8 = 17$ vertices, 16 edges. $C = 2 \cdot 4 + 2 \cdot 4 = 16$. $B = 16 + 4 \cdot 4 = 32$.

Arrangement: $0, z_1, z_2, z_3, 1, z_4, z_5, z_6, 2, y_1, y_2, y_3, y_4, y'_1, y'_2, y'_3, y'_4$. Path costs: $4 + 4 + 4 + 4 = 16 = C$. Additional cost from spacing $= 4 \cdot L_G(f) = 4 \cdot 4 = 16$. Total: $32 = B$. ✓

4 Set and Domination Reductions

4.1 Dominating Set → Min-Max Multicenter

Theorem 4.1.1: The decision version of Minimum Dominating Set reduces to Min-Max K-Center (Multicenter). Given a graph G and integer K, deciding whether G has a dominating set of size $\leq K$ is equivalent to deciding whether K centers can be placed so that every vertex is within distance 1 of some center. Reference: Garey & Johnson (1979), ND50.

Proof: *Construction.* Given a Dominating Set instance $(G = (V, E), K)$:

1. Use the same graph G as the metric space (shortest-path distances).
2. Set the number of centers $= K$.
3. Set the maximum distance bound $B = 1$.
4. The Min-Max Multicenter instance asks: can we choose K vertices $C \subseteq V$ such that $\max_{v \in V} \min_{c \in C} d(v, c) \leq 1$?

Correctness.

($\Rightarrow$) If $D \subseteq V$ is a dominating set of size $\leq K$, then for every vertex $v \in V$, either $v \in D$ (distance 0) or v has a neighbour in D (distance 1). Place centers at D; the maximum distance is ≤ 1. ✓

($\Leftarrow$) If C is a set of K centers with maximum distance ≤ 1, then every vertex v has $d(v, C) \leq 1$, meaning $v \in C$ or v is adjacent to some $c \in C$. Thus C is a dominating set of size K. ✓

Solution extraction. The center set C is directly the dominating set.

Model alignment note. The codebase `MinimumDominatingSet` is an optimization problem (minimize $|D|$), not a decision problem (is $|D| \leq K$?). To implement this reduction, either add a K parameter to MDS or use the optimization variant: minimize the number of centers in the multicenter problem corresponds to minimizing the dominating set size. The reduction then becomes: opt-MDS(G) = opt-MinMax-Multicenter$(G, B = 1)$. □

Overhead.

Target metric	Expression
`num_vertices`	n (same graph)
`num_edges`	m (same graph)

Example. $G = P_4$ (path 0–1–2–3), $K = 2$.

Dominating set: $D = \{1, 2\}$ — vertex 0 is adjacent to 1, vertex 3 is adjacent to 2. Size $= 2 \leq K$. ✓

Multicenter: centers at $\{1, 2\}$. Max distance: $d(0, 1) = 1$, $d(3, 2) = 1$. Max $= 1 \leq B$. ✓

4.2 Dominating Set → Min-Sum Multicenter

Theorem 4.2.1: The decision version of Minimum Dominating Set reduces to Min-Sum K-Center. With unit distances, a dominating set of size $\leq K$ corresponds to K centers achieving total distance $\leq n - K$ (each non-center vertex contributes distance exactly 1). Reference: Garey & Johnson (1979), ND51.

Proof: *Construction.* Given a Dominating Set instance $(G = (V, E), K)$:

1. Use the same graph G.
2. Set the number of centers $= K$.
3. Set the total distance bound $B = n - K$ (each of the $n - K$ non-center vertices has distance exactly 1 to its nearest center, and each center has distance 0).
4. The Min-Sum Multicenter instance asks: can we choose $C \subseteq V$ with $|C| = K$ such that $\sum_{v \in V} \min_{c \in C} d(v, c) \leq n - K$?

Correctness.

($\Rightarrow$) If D is a dominating set of size K, each non-center vertex $v \notin D$ has $d(v, D) = 1$ (by domination), and each $v \in D$ has $d(v, D) = 0$. Total $= 0 \cdot K + 1 \cdot (n - K) = n - K \leq B$. ✓

($\Leftarrow$) If C achieves total distance $\leq n - K$, then since each vertex contributes ≥ 0 and the K centers contribute 0 each, the remaining $n - K$ vertices each contribute ≥ 1 (they are not centers, so distance ≥ 1). Total $\geq n - K$. Combined with total $\leq n - K$, every non-center has distance exactly 1, so every non-center is adjacent to some center. Thus C is a dominating set. ✓

Solution extraction. The center set C is the dominating set.

Model alignment note. Same as Theorem 4.1.1 — needs decision-variant MDS or optimization-variant mapping. □

Overhead.

Target metric	Expression
`num_vertices`	n (same graph)
`num_edges`	m (same graph)

Example. $G = P_4$, $K = 2$.

Dominating set $D = \{1, 2\}$: total distance $= d(0, \{1, 2\}) + d(1, \{1, 2\}) + d(2, \{1, 2\}) + d(3, \{1, 2\}) = 1 + 0 + 0 + 1 = 2 = n - K$. ✓

4.3 Exact Cover by 3-Sets → Acyclic Partition

Status: OPEN. The construction below was found to be **incorrect** by computational verification (see `verify-reductions/verify_all.py`). The 2-cycle approach to block incompatible pairs inadvertently creates quotient-graph cycles between distinct groups, violating the acyclicity constraint. The original Garey & Johnson construction (ND15, citing an unpublished manuscript) is not available in the public literature. A correct reduction requires a fundamentally different encoding of the covering constraint.

Theorem 4.3.1: Exact Cover by 3-Sets (X3C) reduces to Acyclic Partition. Reference: Garey & Johnson (1979), ND15 (citing "Garey and Johnson, unpublished result"). The construction below is an **attempted** reduction that fails on computational verification. It is included for documentation of the approach and its failure mode.

Proof: *Construction.* Given an X3C instance: universe $U = \{u_1, ..., u_{3q}\}$, collection $\mathcal{C} = \{C_1, ..., C_s\}$ where each $|C_j| = 3$.

1. **Vertices.** Create one vertex v_i for each element $u_i \in U$, with weight $w(v_i) = 1$. Total: $3q$ vertices.
2. **Conflict arcs (2-cycles).** For each pair (i, j) with $i < j$: if no subset $C_k \in \mathcal{C}$ contains both u_i and u_j, add both arcs (v_i, v_j) and (v_j, v_i), each with cost 1. This directed 2-cycle makes the pair's induced subgraph cyclic, preventing v_i and v_j from belonging to the same group.
3. **Compatibility arcs.** For each pair (i, j) with $i < j$: if some $C_k \in \mathcal{C}$ contains both u_i and u_j, add only the forward arc (v_i, v_j) with cost 1. The pair can share a group without creating a cycle.
4. **Triple-exclusion arcs.** For each triple (i, j, k) with $i < j < k$: if all three pairs (i, j), (j, k), (i, k) are compatible (each shares a subset in $\mathcal{C}$) but $\{u_i, u_j, u_k\} \notin \mathcal{C}$, add the arc (v_k, v_i) with cost 1. Together with the existing forward arcs (v_i, v_j) and (v_j, v_k), this creates the directed 3-cycle $v_i \to v_j \to v_k \to v_i$, preventing all three from occupying the same group.
5. **Parameters.** Weight bound $B = 3$. Let A denote the total number of arcs constructed. Set the inter-group cost bound $K = A - 3q$.

Justification of the cost bound. For a valid subset $C_\ell = \{u_a, u_b, u_c\} \in \mathcal{C}$ with $a < b < c$: the three pairs (a, b), (b, c), (a, c) are all compatible (they share C_ℓ), and the triple is in $\mathcal{C}$, so no triple-exclusion arc exists. The intra-group arcs are exactly (v_a, v_b), (v_b, v_c), (v_a, v_c) — three forward arcs forming a DAG. Thus each valid group contributes exactly 3 intra-group arcs. With q groups of 3: total intra-group arcs $= 3q$, inter-group cost $= A - 3q = K$.

Correctness.

($\Rightarrow$) Suppose $\mathcal{C}$ has an exact cover $\{C_{j_1}, ..., C_{j_q}\}$. Partition vertices into q groups, one per cover subset. Each group $\{v_a, v_b, v_c\}$ (with $a < b < c$) has:

- Weight $3 \leq B$. ✓
- Induced subgraph: arcs (v_a, v_b), (v_b, v_c), (v_a, v_c) — a DAG (all arcs go from smaller to larger index). ✓
- Intra-group arc count: exactly 3.

The quotient graph contracts each group to a single node. All inter-group arcs go from groups containing smaller-indexed elements to groups with larger-indexed elements (since all arcs in the constructed graph either go from v_i to v_j with $i < j$, or are reverse arcs in 2-cycles, and reverse arcs only exist for incompatible pairs which are in different groups). Thus the quotient graph is acyclic. Inter-group cost $= A - 3q = K$. ✓

($\Leftarrow$) Suppose a valid acyclic partition exists with weight bound $B = 3$ and inter-group cost $\leq K = A - 3q$.

Step 1: All groups have exactly 3 elements. Each vertex has weight 1, so each group has ≤ 3 elements. The total weight is $3q$. Suppose the partition has $q + r$ groups for some $r \geq 0$. The total number of intra-group arcs is at most:

- 3 arcs per group of size 3, 1 arc per group of size 2, 0 arcs per group of size 1.

To distribute $3q$ elements among $q + r$ groups of size ≤ 3: at least r groups have size ≤ 2. Replacing a group of size 3 with groups of size 2 and 1 (or two groups of smaller size) reduces the intra-group arc count by at least 2 (from 3 to at most 1). So the total intra-group count is at most $3q - 2r$. The inter-group cost is at least $A - (3q - 2r) = K + 2r$. If $r > 0$, this exceeds K, contradicting the bound. Hence $r = 0$: exactly q groups, each of size 3.

Step 2: Each group is a valid subset. For any group $\{v_i, v_j, v_k\}$ with $i < j < k$:

- The induced subgraph must be acyclic. A 2-cycle between any pair would make it cyclic, so no 2-cycle exists among (i, j), (j, k), (i, k). By construction, no 2-cycle means each pair is compatible (shares a subset in $\mathcal{C}$).
- No 3-cycle exists either. By construction, the absence of a 3-cycle means: either some pair is incompatible (already excluded) or the triple $\{u_i, u_j, u_k\} \in \mathcal{C}$.

Since all pairs are compatible and the triple has no 3-cycle, $\{u_i, u_j, u_k\} \in \mathcal{C}$.

Step 3: Exact cover. The q groups of 3 are disjoint, cover all of U, and each corresponds to a subset in $\mathcal{C}$. ✓

Solution extraction. Each group of 3 element-vertices directly identifies a subset in the exact cover. □

Overhead.

Target metric	Expression
`num_vertices`	$3q$
`num_arcs`	$\leq 2\binom{3q}{2} + \binom{3q}{3}$ (at most 2 arcs per pair plus 1 per incompatible triple)

Example. $U = \{1, 2, 3, 4, 5, 6\}$, $\mathcal{C} = \{\{1, 2, 3\}, \{1, 2, 4\}, \{4, 5, 6\}\}$, $q = 2$.

Valid exact cover: $\{\{1, 2, 3\}, \{4, 5, 6\}\}$.

Arcs constructed:

- Compatible pairs (forward arcs only): $(1, 2), (1, 3), (2, 3)$ (share $\{1, 2, 3\}$); $(1, 4), (2, 4)$ (share $\{1, 2, 4\}$); $(4, 5), (4, 6), (5, 6)$ (share $\{4, 5, 6\}$). Count: 8 arcs.
- Incompatible pairs (2-cycles): $(1, 5), (5, 1), (1, 6), (6, 1), (2, 5), (5, 2), (2, 6), (6, 2), (3, 4), (4, 3), (3, 5), (5, 3), (3, 6), (6, 3)$. Count: 14 arcs.
- Triple-exclusion: triple $\{1, 2, 4\}$ is pairwise compatible and $\{1, 2, 4\} \in \mathcal{C}$, so no exclusion arc needed.

Total arcs: $A = 8 + 14 = 22$. Cost bound: $K = 22 - 6 = 16$.

Partition $\{\{1, 2, 3\}, \{4, 5, 6\}\}$: Group $\{1, 2, 3\}$: intra-group arcs $(1, 2), (2, 3), (1, 3)$ — DAG, cost 3. Group $\{4, 5, 6\}$: intra-group arcs $(4, 5), (5, 6), (4, 6)$ — DAG, cost 3. Inter-group cost: $22 - 6 = 16 = K$. Quotient: all inter-group arcs go forward. Acyclic. ✓

5 Feedback Set Reductions

5.1 Vertex Cover → Partial Feedback Edge Set

Theorem 5.1.1: Vertex Cover reduces to Partial Feedback Edge Set (PFES). Given a graph $G = (V, E)$ and budget K, we construct a graph H with a control edge per vertex and a 6-cycle per edge of G, such that deleting $\leq K$ edges from H to break all cycles of length ≤ 6 is equivalent to finding a vertex cover of size $\leq K$ in G. The control-edge construction ensures that only control edges need to be deleted in any optimal solution. Reference: Garey & Johnson (1979), GT12; based on the framework of Yannakakis (1978).

Proof: *Construction.* Given a Vertex Cover instance $(G = (V, E), K)$ with $n = |V|$ and $m = |E|$.

1. **Control vertices and edges.** For each vertex $v \in V$, add a new vertex r_v and a control edge $e_v^* = (v, r_v)$.
2. **Edge gadgets.** For each edge $(u, w) \in E$, add two new vertices s_{uw} and p_{uw} and the following four edges:
   - (r_u, s_{uw}) and (s_{uw}, r_w) — connecting control vertices through s_{uw}.
   - (u, p_{uw}) and (p_{uw}, w) — a path of length 2 replacing the original edge (u, w).

   This creates the 6-cycle:
   $$u - r_u - s_{uw} - r_w - w - p_{uw} - u$$
   whose six edges are: e_u^*, (r_u, s_{uw}), (s_{uw}, r_w), e_w^*, (w, p_{uw}), (p_{uw}, u).
3. **Parameters.** Set the cycle-length bound $L = 6$ and the edge-deletion budget $K' = K$.

Vertex and edge counts of H:
- Vertices: n (original) $+n$ (control vertices r_v) $+2m$ (two gadget vertices s_{uw}, p_{uw} per edge) $= 2n + 2m$.
- Edges: n (control edges) $+4m$ (four gadget edges per original edge) $= n + 4m$.

Cycle analysis. We verify that H has no cycles of length less than 6.

No 3-cycles. A 3-cycle would require three mutually adjacent vertices. The vertices r_v connect only to v and to s-vertices. The s-vertices connect only to two r-vertices. The p-vertices connect only to two original vertices. No three of these can form a triangle: any cycle through r_u must alternate between r_u's neighbours (u and the s-vertices), and no two neighbours of r_u are adjacent to each other (u is not adjacent to any s-vertex in H, since the s-vertex edges go to r-vertices, not original vertices; and s-vertices are not adjacent to each other).

No 4-cycles. A 4-cycle through r_u would need two of r_u's neighbours to be at distance 2 from each other. The neighbours of r_u are u and the various s_{uw}. For two neighbours s_{uw_1} and s_{uw_2}: a common neighbour would have to be some r_v with $\left(r_v, s_{uw_1}\right)$ and $\left(r_v, s_{uw_2}\right)$ both edges; this requires $v = w_1$ and $v = w_2$, so $w_1 = w_2$, contradicting distinctness. For u and s_{uw}: a common neighbour of u (besides r_u) is some $p_{uw'}$, and a common neighbour of s_{uw} (besides r_u) is r_w. These are never the same vertex.

No 5-cycles. The graph H is bipartite-like between "level-0" vertices (original v and p-vertices) and "level-1" vertices (r_v and s-vertices). Every edge connects a level-0 vertex to a level-1 vertex:
- (v, r_v): level-0 to level-1. ✓
- (r_u, s_{uw}): level-1 to level-1. This breaks the bipartite structure.

Since the bipartite argument does not hold exactly, we verify directly. A 5-cycle must have odd length; but examine the vertex types on a hypothetical 5-cycle. Each s-vertex has degree 2 (connected to r_u and r_w), each p-vertex has degree 2 (connected to u and w), each r_v has degree $d(v) + 1$ (connected to v and $d(v)$ vertices s_{vw}). A 5-cycle cannot pass through only r and s vertices (those form a bipartite subgraph with all edges between r-vertices and s-vertices, yielding only even cycles). Including an original vertex u: from u, the cycle can go to r_u or to some p_{uw}. From r_u, it can go to $s_{uw'}$ or back to u. Following the

path $u - r_u - s_{uw_1} - r_{w_1} - \ldots$: after r_{w_1}, the cycle can go to w_1 or to $s_{w_1 w_2}$. Going to $w_1 - p_{w_1 u} - u$ gives length 6, not 5. Going to $s_{w_1 w_2} - r_{w_2} - \ldots$ extends beyond 5. No 5-cycle is achievable.

Therefore, the only cycles of length ≤ 6 are the 6-cycles, exactly one per edge of G.

Dominance of control edges. We prove that any optimal PFES solution can be converted to one using only control edges, without increasing its size.

Claim: For each non-control edge e in some solution F that breaks a 6-cycle for edge (u, w), replacing e with the control edge e_u^* (or e_w^*) yields a solution of size $\leq |F|$.

Proof: The edge e participates in exactly one 6-cycle (the one for edge (u, w)). This is because:

- (r_u, s_{uw}) appears only in the 6-cycle for (u, w).
- (s_{uw}, r_w) appears only in the 6-cycle for (u, w).
- (u, p_{uw}) appears only in the 6-cycle for (u, w).
- (p_{uw}, w) appears only in the 6-cycle for (u, w).

In contrast, the control edge e_u^* appears in every 6-cycle for an edge incident to u: there are $d(u)$ such cycles. Replacing e with e_u^* breaks the cycle for (u, w) (which e was breaking) and additionally breaks all other cycles through e_u^*. Some of these may have previously required other edges in F to be deleted; those edges in F are now redundant and can be removed, reducing $|F|$. At worst, no other edges become redundant, and $|F|$ stays the same. □

Applying this replacement repeatedly transforms F into a solution $F' \subseteq \{e_v^* : v \in V\}$ with $|F'| \leq |F| \leq K$.

Correctness.

($\Rightarrow$) If C is a vertex cover of G with $|C| \leq K$, delete $F = \{e_v^* : v \in C\}$, giving $|F| = |C| \leq K$. For each edge $(u, w) \in E$: since C covers (u, w), at least one of u, w is in C, so e_u^* or e_w^* is in F. The 6-cycle u–r_u–s_{uw}–r_w–w–p_{uw}–u passes through both e_u^* and e_w^*; deleting either one breaks the cycle. All 6-cycles are broken. ✓

($\Leftarrow$) If F is a PFES solution with $|F| \leq K$ for $(H, L = 6)$, convert it to a control-edge-only solution F' with $|F'| \leq K$ (by the dominance argument above). Define $C = \{v \in V : e_v^* \in F'\}$. For each edge $(u, w) \in E$: the 6-cycle for (u, w) must be broken, so at least one of e_u^*, e_w^* is in F', meaning $u \in C$ or $w \in C$. Thus C is a vertex cover with $|C| = |F'| \leq K$. ✓

Solution extraction. Given a PFES solution F, apply the dominance replacement to obtain F' consisting of control edges only. The set $C = \{v : e_v^* \in F'\}$ is a vertex cover of G. □

Overhead.

Target metric	Expression
`num_vertices`	$2n + 2m$
`num_edges`	$n + 4m$

Example. $G = P_3$ (path 0–1–2, edges $e_1 = (0, 1)$, $e_2 = (1, 2)$), $K = 1$.

Construction:

- Control edges: $e_0^* = (0, r_0)$, $e_1^* = (1, r_1)$, $e_2^* = (2, r_2)$.
- Edge $(0, 1)$: vertices s_{01}, p_{01}. Edges: $(r_0, s_{01}), (s_{01}, r_1), (0, p_{01}), (p_{01}, 1)$.
- Edge $(1, 2)$: vertices s_{12}, p_{12}. Edges: $(r_1, s_{12}), (s_{12}, r_2), (1, p_{12}), (p_{12}, 2)$.

Vertices: $2 \cdot 3 + 2 \cdot 2 = 10$. Edges: $3 + 4 \cdot 2 = 11$.

6-cycles:

- For $(0, 1)$: 0–r_0–s_{01}–r_1–1–p_{01}–0 (length 6).
- For $(1, 2)$: 1–r_1–s_{12}–r_2–2–p_{12}–1 (length 6).

Vertex cover $C = \{1\}$: delete $e_1^* = (1, r_1)$. Both 6-cycles pass through r_1 (the first via edge (s_{01}, r_1), which precedes e_1^* in the cycle; the second via edge (r_1, s_{12}), which follows e_1^*). Deleting e_1^* removes r_1 from both cycles, breaking them. Total deletions $= 1 = K$. ✓

No shorter cycles: the 10-vertex graph has r-vertices connecting only to original vertices and s-vertices, s-vertices connecting only to r-vertices, and p-vertices connecting only to original vertices. No 3-, 4-, or 5-cycles can form. ✓

6 Satisfiability Reductions

6.1 Satisfiability → Non-Tautology

Theorem 6.1.1: Satisfiability reduces to Non-Tautology via De Morgan duality. Given a CNF formula φ, its negation $\neg\varphi$ is a DNF formula E. The formula φ is satisfiable if and only if $E = \neg\varphi$ is not a tautology: a satisfying assignment for φ is exactly a falsifying assignment for $\neg\varphi$. Reference: Cook (1971); Garey & Johnson (1979), LO8.

Proof: *Construction.* Given a Satisfiability instance: CNF formula $\varphi = C_1 \wedge C_2 \wedge ... \wedge C_m$ over variables $U = \{x_1, ..., x_n\}$, where each clause $C_j = \left(\ell_{j,1} \vee \ell_{j,2} \vee ... \vee \ell_{j,k_j}\right)$.

1. Apply De Morgan's laws to negate φ:
$$\neg\varphi = \neg C_1 \vee \neg C_2 \vee ... \vee \neg C_m$$
2. Negate each clause: $\neg C_j = \neg\left(\ell_{j,1} \vee ... \vee \ell_{j,k_j}\right) = \left(\neg\ell_{j,1} \wedge ... \wedge \neg\ell_{j,k_j}\right)$.
3. The result is a DNF formula $E = D_1 \vee D_2 \vee ... \vee D_m$ where each disjunct $D_j = \left(\neg\ell_{j,1} \wedge ... \wedge \neg\ell_{j,k_j}\right)$ is the bitwise negation of clause C_j.
4. Output the Non-Tautology instance (U, E) with the same variable set and m disjuncts.

Correctness.

($\Rightarrow$) If φ is satisfiable with assignment α, then $\varphi(\alpha) = \top$, so $E(\alpha) = (\neg\varphi)(\alpha) = \neg\top = \bot$. Assignment α falsifies E, so E is not a tautology. ✓

($\Leftarrow$) If E is not a tautology, there exists assignment α with $E(\alpha) = \bot$. Then $(\neg\varphi)(\alpha) = \bot$, so $\varphi(\alpha) = \top$. Assignment α satisfies φ. ✓

Solution extraction. The falsifying assignment for E is the satisfying assignment for φ. No transformation needed — identity extraction. □

Overhead.

Target metric	Expression
`num_vars`	n (same variables)
`num_disjuncts`	m (one per clause)

Example. $\varphi = (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_2 \vee \neg x_3)$ over $\{x_1, x_2, x_3\}$.

Negation: $E = (\neg x_1 \wedge x_2) \vee (x_1 \wedge \neg x_2 \wedge \neg x_3) \vee (\neg x_2 \wedge x_3)$.

Assignment $\alpha = (x_1 = \top, x_2 = \top, x_3 = \bot)$:
- φ: $(\top \vee \bot) \wedge (\bot \vee \top \vee \bot) \wedge (\top \vee \top) = \top \wedge \top \wedge \top = \top$. Satisfies φ. ✓
- E: $(\bot \wedge \top) \vee (\top \wedge \bot \wedge \top) \vee (\bot \wedge \bot) = \bot \vee \bot \vee \bot = \bot$. Falsifies E. ✓

7 References

1. Garey, M. R. and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness.* W.H. Freeman and Company.

2. Garey, M. R., Johnson, D. S., and Stockmeyer, L. (1976). "Some simplified NP-complete graph problems." *Theoretical Computer Science* 1(3), pp. 237–267.

3. Gavril, F. (1977). "Some NP-complete problems on graphs." *Proc. 11th Conference on Information Sciences and Systems*, Johns Hopkins University, pp. 91–95.

4. Karp, R. M. (1972). "Reducibility among combinatorial problems." In *Complexity of Computer Computations*, Plenum Press, pp. 85–103.

5. Yannakakis, M. (1978). "Node- and edge-deletion NP-complete problems." *Proc. 10th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 253–264.